A Project Report on

AI-POWERED SMART HEALTHCARE APPOINTMENT AND ASSISTANCE SYSTEM

B. TECH

in

Department of CSE(AI & ML)

Submitted by

D.AVANTHI (237R1A66L9)

TABLE OF CONTENTS

ABSTRACT

The AI Healthcare Appointment System is a web-based application designed to simplify the process of booking medical appointments and recommending suitable doctors based on patient symptoms. The system provides separate modules for Patients, Doctors, and Administrators. Patients can register, log in, describe their symptoms, receive AI-based doctor recommendations, and book appointments online. Doctors can manage appointments and update their availability, while administrators can manage doctors, patients, and system records. The system uses Machine Learning techniques to analyze symptoms and recommend the most appropriate medical specialist. This project improves healthcare accessibility, reduces appointment scheduling time, and enhances the overall patient experience.

INTRODUCTION

1.1 Introduction to AI Powered HealthCare System:

Healthcare services often require patients to manually search for suitable doctors and schedule appointments, which can be time-consuming and inefficient. Many patients are unsure about which specialist to consult based on their symptoms. The AI Healthcare Appointment System addresses this problem by providing an intelligent platform that analyzes symptoms and recommends the appropriate doctor specialization. The system also enables online appointment booking, appointment management, and healthcare service automation. By integrating Artificial Intelligence and Machine Learning, the system improves decision-making and streamlines healthcare processes.

CHAPTER - 2

LITERATURE SURVEY

2.1 Existing System

The existing healthcare appointment process is largely manual and time-consuming. Patients must visit hospitals or clinics physically, contact hospital staff, or make phone calls to schedule appointments. Finding the appropriate specialist often depends on personal knowledge or recommendations from others, which may lead to delays in receiving proper medical care. Most traditional systems do not provide intelligent doctor recommendations based on symptoms, making it difficult for patients to identify the right healthcare professional. Additionally, appointment records, doctor availability, and patient information are often managed manually or through separate systems, resulting in inefficiencies and increased administrative workload. These limitations can lead to long waiting times, poor appointment management, reduced accessibility, and inconvenience for both patients and healthcare providers.

2.2 Disadvantages of the Existing System

- Manual appointment booking process.
- Difficulty in finding the appropriate specialist.
- Time-consuming and inefficient healthcare management.
- Lack of AI-based doctor recommendations.
- Increased waiting time for patients.
- Limited accessibility and convenience.
- Poor appointment tracking and management.
- Higher administrative workload.
- Possibility of human errors in record maintenance.
- Lack of centralized healthcare information management

CHAPTER – 3

PROJECT OVERVIEW

3.1 Project Overview

The AI Healthcare Appointment System is a web-based healthcare management application designed to streamline the process of finding suitable doctors and booking medical appointments. The system integrates Artificial Intelligence (AI) and Machine Learning (ML) technologies to analyze patient symptoms and recommend the most appropriate doctor specialization. It provides separate modules for Patients, Doctors, and Administrators, ensuring efficient management of healthcare services.

Patients can register, log in, enter their symptoms, receive AI-based doctor recommendations, view available doctors, and book appointments online. Doctors can manage their schedules, view patient appointments, and update their availability. Administrators can monitor and manage doctors, patients, appointments, and overall system operations through a dedicated dashboard.

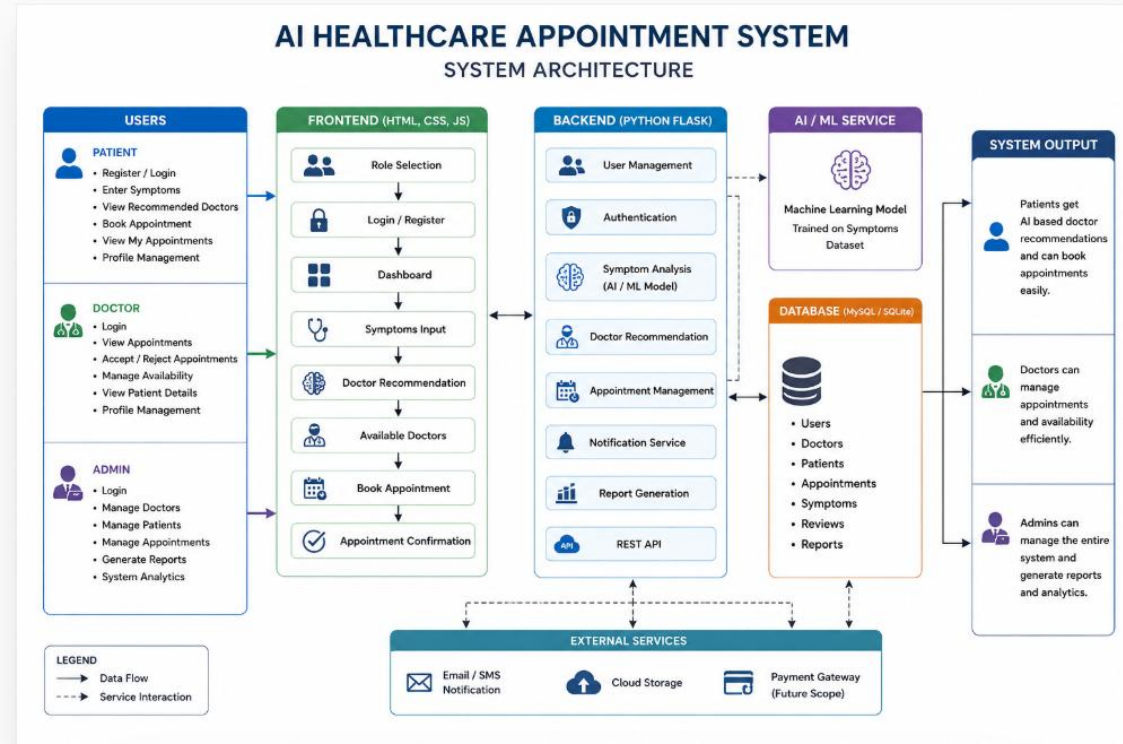
The system aims to reduce the time and effort involved in traditional appointment booking processes while improving healthcare accessibility and patient experience. By leveraging Machine Learning for symptom analysis and doctor recommendation, the platform helps patients quickly connect with the right healthcare specialist. The project demonstrates the practical application of AI in healthcare and provides a modern, efficient, and user-friendly solution for appointment management.

OBJECTIVES

- To develop an intelligent healthcare appointment management system.
- To recommend suitable doctors using Artificial Intelligence.
- To automate appointment scheduling and management.
- To improve healthcare accessibility and convenience.
- To provide separate modules for Patients, Doctors, and Administrators.

System Architecture:

1. Patient → Login → Enter Symptoms → AI Analysis → Doctor Recommendation → Available Doctors → Appointment Booking → Appointment Confirmation
2. Doctor → Login → View Appointments → Manage Availability
3. Admin → Login → Manage Users, Doctors, and Appointments



CHAPTER - 4

IMPLEMENTATION

4.1 Techonologies And Their Implementation

1. HTML (HyperText Markup Language)

Description:
HTML is used to create the structure and layout of web pages.

Implementation in Project:

- Login Page
- Registration Page
- Dashboard
- Symptoms Entry Form
- Doctor Listing Page
- Appointment Booking Page
- Admin Panel

2. CSS (Cascading Style Sheets)

- **Description:**

CSS is used to design attractive, responsive, and user-friendly interfaces.

- **Implementation in Project:**
- Healthcare-themed UI design
- Responsive layouts
- Dashboard cards
- Navigation menus
- Doctor profile cards
- Appointment forms

3.JavaScript

- **Description:**
  JavaScript is used to add interactivity and dynamic functionality to web pages.
- **Implementation in Project:**
- Form validation
- Dynamic doctor listing
- Appointment booking actions
- User interactions
- Dashboard updates
- API requests and responses

---

4. Flask / FastAPI

- **Description:**
  Flask (or FastAPI) is used as the backend framework for handling business logic and APIs.
- **Implementation in Project:**
- User authentication
- Appointment management
- Doctor management
- Patient management
- API development
- Database connectivity
-

---

5. MySQL / MongoDB

- **Description:**
  Database systems are used to store and manage application data.
- **Implementation in Project:**
- Patient records
- Doctor details
- Appointment information
- Login credentials
- System data storage
-

---

6. Machine Learning (Scikit-Learn)

- **Description:**
  Scikit-Learn is a Machine Learning library used to build predictive models.
- **Implementation in Project:**
- Symptom analysis
- Doctor recommendation system
- Model training and prediction
- Classification of symptoms into medical specializations

Example:

```
Symptoms: Fever, Cough
Prediction: General Physician

Symptoms: Chest Pain
Prediction: Cardiologist
```

---

7. Artificial Intelligence (AI)

- **Description:**
  Artificial Intelligence enables the system to provide intelligent recommendations.
- **Implementation in Project:**
- AI-based symptom analysis
- Smart doctor recommendation
- Healthcare decision support
- Automated patient guidance
- ---

8. Generative AI (LLM API)

- **Description:**
  Large Language Models (LLMs) such as Gemini or OpenAI provide natural language understanding and generation.
- **Implementation in Project:**
- Medical report simplification
- Patient-friendly explanations
- Health guidance generation
- AI-powered healthcare assistance

MODULES

Patient Module

- Registration
- Login
- Symptom Analysis
- Doctor Recommendation
- Appointment Booking
- Appointment Tracking
- Profile Management

Doctor Module

- Login
- View Appointments
- Manage Availability
- View Patient Details

Admin Module

- Login
- Manage Doctors
- Manage Patients
- Manage Appointments
- Generate Reports

AI Recommendation Module

- Analyze Symptoms
- Predict Doctor Specialization

- Provide Recommendations

MACHINE LEARNING MODEL

The Machine Learning model is trained using a symptom dataset containing symptoms and corresponding doctor specializations. The model learns symptom patterns and predicts the most suitable specialist based on user input.

Example:

Symptoms: Fever, Cough

Prediction: General Physician

Symptoms: Chest Pain

Prediction: Cardiologist

Symptoms: Skin Rash

Prediction: Dermatologist

4.2 System Requirements

Hardware Requirements

Component	Requirement
Processor	Intel Core i3 / i5 or above
RAM	4 GB Minimum (8 GB Recommended)
Hard Disk	20 GB Free Space
System Type	64-bit Operating System
Internet Connection	Required
Display	1366 × 768 Resolution or Higher

Software Requirements

Software	Purpose
Windows 10/11	Operating System
Visual Studio Code	Code Editor
Python 3.10+	Backend Development
Flask / FastAPI	Backend Framework
HTML5	Frontend Structure
CSS3	User Interface Design
JavaScript	Frontend Functionality
MySQL / MongoDB	Database Management
Scikit-Learn	Machine Learning Model
Pandas	Dataset Processing
Joblib	Model Saving and Loading
Google Chrome	Application Testing
Git & GitHub	Version Control

4.3 CODE:-

```
"""
MediCare AI - Flask Backend
Healthcare Appointment System with AI-Powered Symptom Analysis
"""

from flask import Flask, request, jsonify
from flask_cors import CORS
from flask_sqlalchemy import SQLAlchemy
from flask_bcrypt import Bcrypt
from flask_jwt_extended import JWTManager, create_access_token, jwt_required
import os

# Initialize Flask app
app = Flask(__name__)
CORS(app)

# Configuration
app.config['SECRET_KEY'] = 'medicare-ai-secret-key-2024'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///medicare.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

# Initialize extensions
db = SQLAlchemy(app)
bcrypt = Bcrypt(app)
jwt = JWTManager(app)

# ==================== MODELS ====================

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    phone = db.Column(db.String(20), nullable=False)
    password = db.Column(db.String(200), nullable=False)
    role = db.Column(db.String(20), default='patient')
    created_at = db.Column(db.DateTime, default=db.func.now())
```

```
class Doctor(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    specialization = db.Column(db.String(100), nullable=False)
    experience = db.Column(db.Integer, default=0)
    rating = db.Column(db.Float, default=0.0)
    fee = db.Column(db.Integer, default=0)
    image = db.Column(db.String(200))
    about = db.Column(db.Text)
    available = db.Column(db.Boolean, default=True)

class Appointment(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    patient_id = db.Column(db.Integer, db.ForeignKey('user.id'))
    doctor_id = db.Column(db.Integer, db.ForeignKey('doctor.id'))
    date = db.Column(db.String(20), nullable=False)
    time = db.Column(db.String(20), nullable=False)
    status = db.Column(db.String(20), default='pending')
    symptoms = db.Column(db.Text)
    created_at = db.Column(db.DateTime, default=db.func.now())

# ==================== ROUTES ====================

# Health check
@app.route('/api/health', methods=['GET'])
def health_check():
    return jsonify({'status': 'healthy', 'message': 'MediCare AI API is running'})

# Auth routes
@app.route('/api/auth/register', methods=['POST'])
def register():
    data = request.get_json()

    if User.query.filter_by(email=data['email']).first():
        return jsonify({'error': 'Email already registered'}), 400

    hashed_password = bcrypt.generate_password_hash(data['password']).decode('utf-8')
    user = User(
```

```
        name=data['name'],
        email=data['email'],
        phone=data['phone'],
        password=hashed_password,
        role='patient'
    )

    db.session.add(user)
    db.session.commit()

    return jsonify({'message': 'User registered successfully'}), 201

@app.route('/api/auth/login', methods=['POST'])
def login():
    data = request.get_json()
    user = User.query.filter_by(email=data['email']).first()

    if user and bcrypt.check_password_hash(user.password, data['password']):
        access_token = create_access_token(identity={'id': user.id, 'role': user.role})
        return jsonify({
            'access_token': access_token,
            'user': {
                'id': user.id,
                'name': user.name,
                'email': user.email,
                'role': user.role
            }
        })

    return jsonify({'error': 'Invalid credentials'}), 401

# Doctor routes
@app.route('/api/doctors', methods=['GET'])
def get_doctors():
    specialization = request.args.get('specialization')

    if specialization:
        doctors = Doctor.query.filter_by(specialization=specialization).all()
```

```
    else:
        doctors = Doctor.query.all()

    return jsonify([{
        'id': d.id,
        'name': d.name,
        'specialization': d.specialization,
        'experience': d.experience,
        'rating': d.rating,
        'fee': d.fee,
        'image': d.image,
        'about': d.about,
        'available': d.available
    } for d in doctors])

@app.route('/api/doctors/<int:id>', methods=['GET'])
def get_doctor(id):
    doctor = Doctor.query.get_or_404(id)
    return jsonify({
        'id': doctor.id,
        'name': doctor.name,
        'specialization': doctor.specialization,
        'experience': doctor.experience,
        'rating': doctor.rating,
        'fee': doctor.fee,
        'image': doctor.image,
        'about': doctor.about,
        'available': doctor.available
    })

# Appointment routes
@app.route('/api/appointments', methods=['POST'])
@jwt_required()
def create_appointment():
    data = request.get_json()
    appointment = Appointment(
        patient_id=data['patient_id'],
        doctor_id=data['doctor_id'],
```

```
        date=data['date'],
        time=data['time'],
        symptoms=data.get('symptoms', ''),
        status='confirmed'
    )

    db.session.add(appointment)
    db.session.commit()

    return jsonify({'message': 'Appointment created successfully'}), 201

@app.route('/api/appointments/<int:id>', methods=['PUT'])
@jwt_required()
def update_appointment(id):
    appointment = Appointment.query.get_or_404(id)
    data = request.get_json()

    if 'status' in data:
        appointment.status = data['status']
    if 'date' in data:
        appointment.date = data['date']
    if 'time' in data:
        appointment.time = data['time']

    db.session.commit()
    return jsonify({'message': 'Appointment updated successfully'})

@app.route('/api/appointments/patient/<int:patient_id>', methods=['GET'])
@jwt_required()
def get_patient_appointments(patient_id):
    appointments = Appointment.query.filter_by(patient_id=patient_id).all()
    return jsonify([{
        'id': a.id,
        'doctor_id': a.doctor_id,
        'date': a.date,
        'time': a.time,
        'status': a.status,
        'symptoms': a.symptoms
```

```
    } for a in appointments])

# AI Symptom Analysis
@app.route('/api/ml/predict', methods=['POST'])
def predict_specialization():
    data = request.get_json()
    symptoms = data.get('symptoms', '').lower()

    # Symptom to specialization mapping
    symptom_map = {
        'fever': 'General Physician',
        'cold': 'General Physician',
        'cough': 'General Physician',
        'flu': 'General Physician',
        'headache': 'General Physician',
        'chest pain': 'Cardiologist',
        'heart': 'Cardiologist',
        'cardiac': 'Cardiologist',
        'skin': 'Dermatologist',
        'rash': 'Dermatologist',
        'acne': 'Dermatologist',
        'eye': 'Ophthalmologist',
        'vision': 'Ophthalmologist',
        'tooth': 'Dentist',
        'teeth': 'Dentist',
        'stomach': 'Gastroenterologist',
        'abdominal': 'Gastroenterologist',
        'nausea': 'Gastroenterologist'
    }

    for keyword, specialization in symptom_map.items():
        if keyword in symptoms:
            return jsonify({
                'specialization': specialization,
                'confidence': 0.95,
                'symptoms': symptoms
            })
```

```
    return jsonify({
        'specialization': 'General Physician',
        'confidence': 0.5,
        'symptoms': symptoms
    })

# ==================== ADMIN ROUTES ====================

@app.route('/api/admin/doctors', methods=['POST'])
@jwt_required()
def add_doctor():
    data = request.get_json()
    doctor = Doctor(
        name=data['name'],
        specialization=data['specialization'],
        experience=data.get('experience', 0),
        rating=data.get('rating', 0.0),
        fee=data.get('fee', 0),
        image=data.get('image'),
        about=data.get('about'),
        available=True
    )

    db.session.add(doctor)
    db.session.commit()

    return jsonify({'message': 'Doctor added successfully'}), 201

@app.route('/api/admin/doctors/<int:id>', methods=['PUT'])
@jwt_required()
def update_doctor(id):
    doctor = Doctor.query.get_or_404(id)
    data = request.get_json()

    for key in ['name', 'specialization', 'experience', 'rating', 'fee', 'image', 'about', 'available']:
        if key in data:
            setattr(doctor, key, data[key])
```

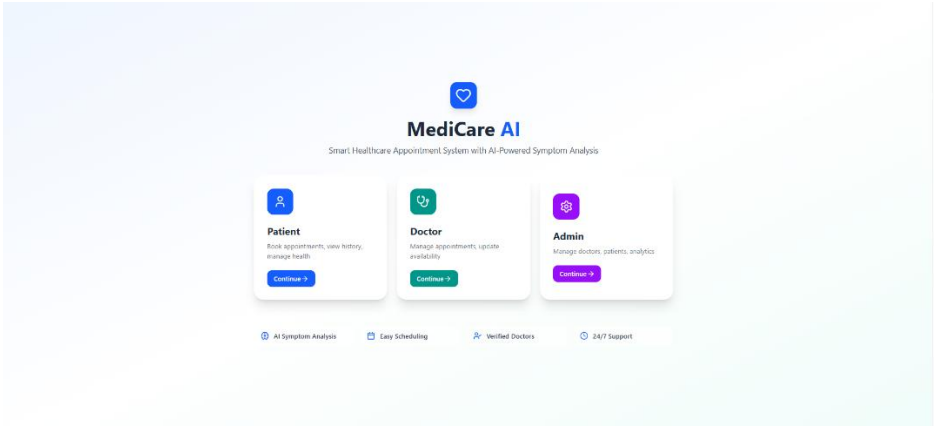
```
    db.session.commit()
    return jsonify({'message': 'Doctor updated successfully'})

@app.route('/api/admin/doctors/<int:id>', methods=['DELETE'])
@jwt_required()
def delete_doctor(id):
    doctor = Doctor.query.get_or_404(id)
    db.session.delete(doctor)
    db.session.commit()
    return jsonify({'message': 'Doctor deleted successfully'})

@app.route('/api/admin/stats', methods=['GET'])
@jwt_required()
def get_stats():
    return jsonify({
        'total_users': User.query.count(),
        'total_doctors': Doctor.query.count(),
        'total_appointments': Appointment.query.count(),
        'pending_appointments': Appointment.query.filter_by(status='pending').count()
    })

# ==================== INIT DATABASE ====================

def init_db():
    with app.app_context():
        db.create_all()

        # Add sample doctors if none exist
        if Doctor.query.count() == 0:
            sample_doctors = [
                Doctor(name='Dr. Sarah Johnson', specialization='General Physician',
                       experience=15, rating=4.9, fee=500,
                       image='https://randomuser.me/api/portraits/women/44.jpg',
                       about='Experienced general physician'),
                Doctor(name='Dr. Michael Chen', specialization='Cardiologist',
                       experience=20, rating=4.8, fee=800,
                       image='https://randomuser.me/api/portraits/men/32.jpg',
                       about='Board-certified cardiologist'),
```

```
            Doctor(name='Dr. Emily Watson', specialization='Dermatologist',
                   experience=12, rating=4.7, fee=600,
                   image='https://randomuser.me/api/portraits/women/65.jpg',
                   about='Specialized in medical dermatology'),
            Doctor(name='Dr. James Wilson', specialization='Ophthalmologist',
                   experience=18, rating=4.9, fee=700,
                   image='https://randomuser.me/api/portraits/men/45.jpg',
                   about='Expert in cataract surgery'),
            Doctor(name='Dr. Lisa Brown', specialization='Dentist',
                   experience=10, rating=4.6, fee=400,
                   image='https://randomuser.me/api/portraits/women/33.jpg',
                   about='General dentist'),
            Doctor(name='Dr. Robert Martinez', specialization='Gastroenterologist',
                   experience=14, rating=4.8, fee=750,
                   image='https://randomuser.me/api/portraits/men/22.jpg',
                   about='Specialist in digestive disorders')
        ]

        for doctor in sample_doctors:
            db.session.add(doctor)

        db.session.commit()
        print("Sample doctors added to database")

if __name__ == '__main__':
    init_db()
    app.run(debug=True, port=5000)
```

CHAPTER – 5

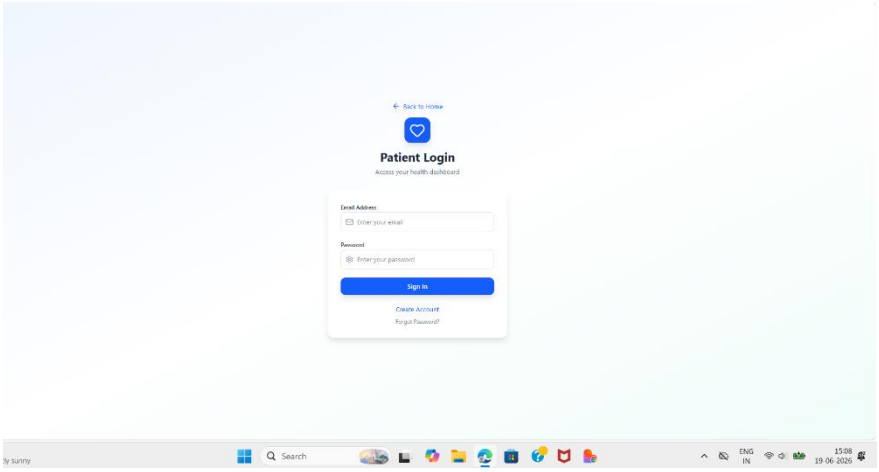
RESULT

Output Screenshots :



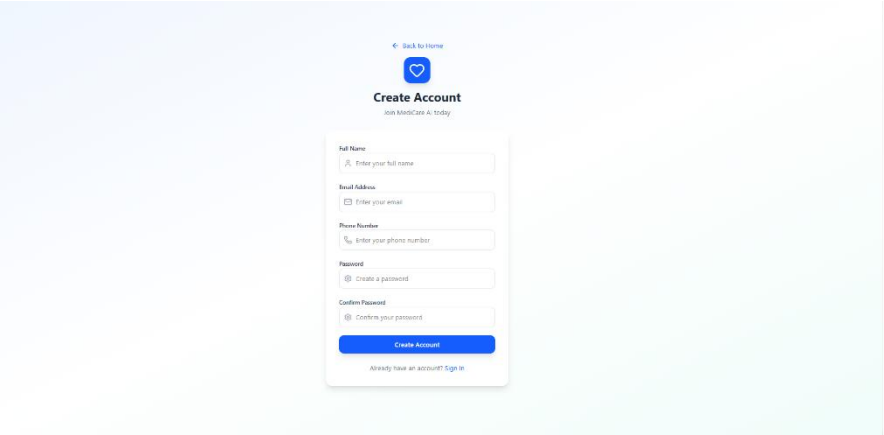
Home Page

This is the home page of MediCare AI. Users can choose whether they want to log in as a Patient, Doctor, or Admin and access the system features."



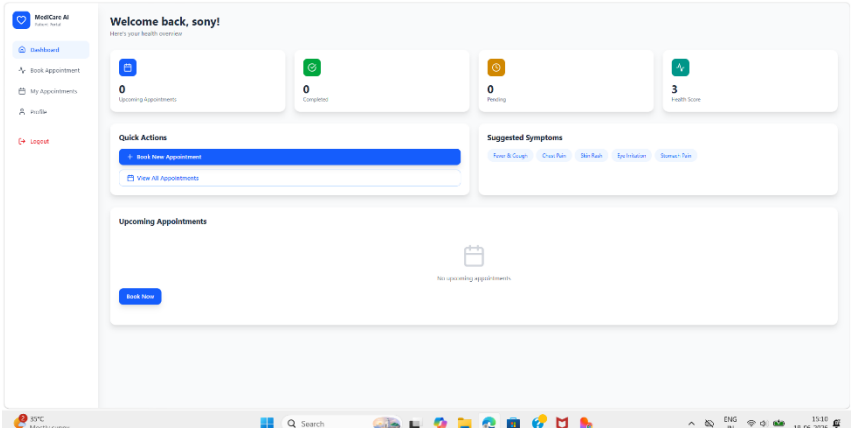
Patient Login

"This is the patient login page where registered users can securely access their accounts using their email and passw



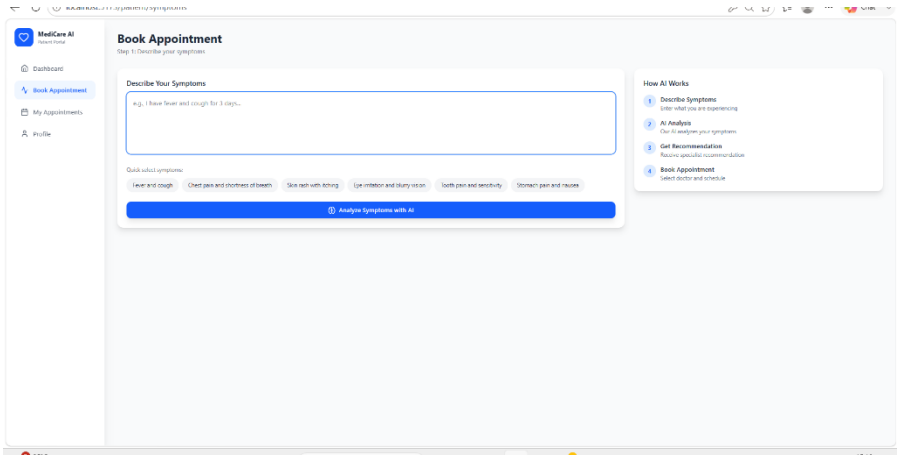
Patient Registration

"This page allows new users to create an account by providing personal details such as name, email, phone number, and password."



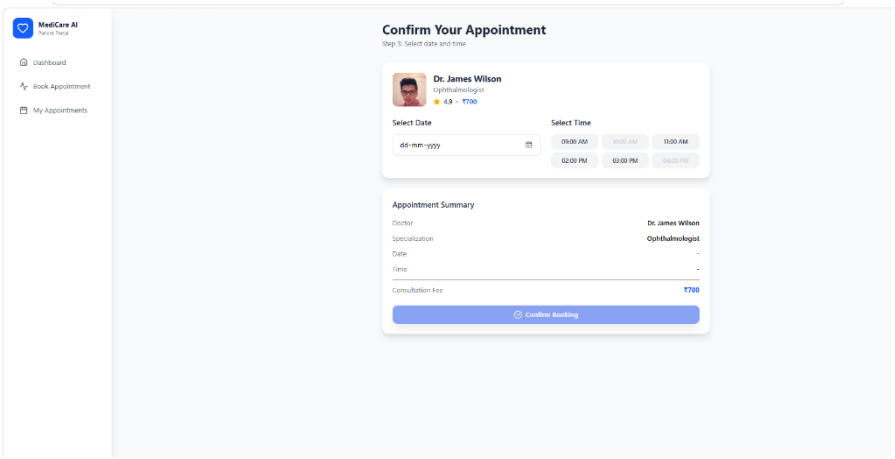
Patient Dashboard

"After login, patients can view their appointment summary, health information, and access different healthcare services from this dashboard."



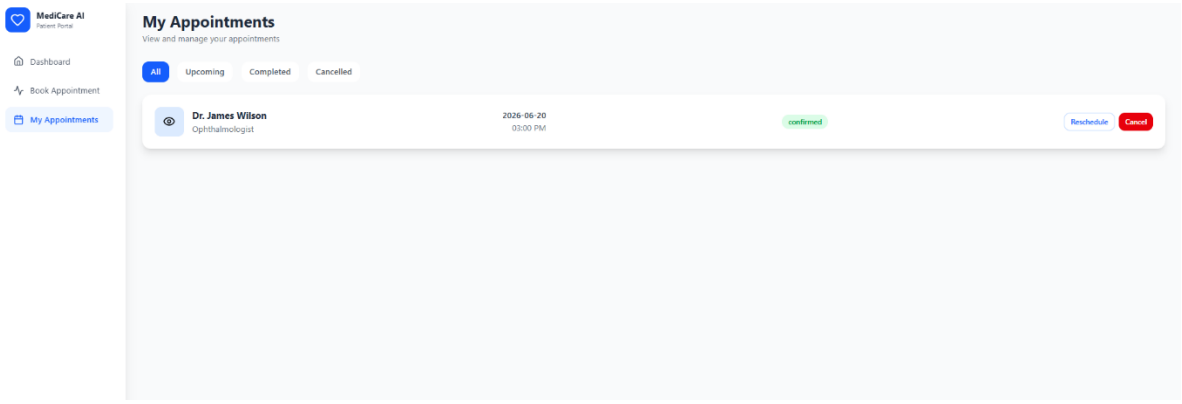
AI Symptom Analysis

"Patients can enter their symptoms here. The AI model analyzes the symptoms and predicts the most suitable medical specialist."



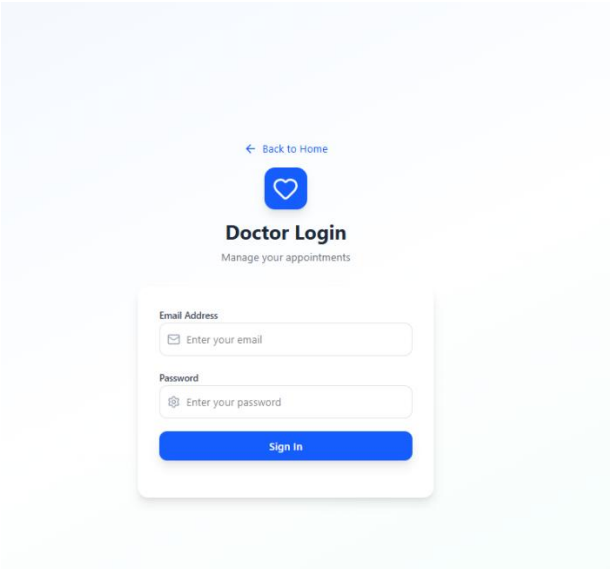
Appointment Booking

"Patients can select an available date and time slot and confirm their appointment with the selected doctor."



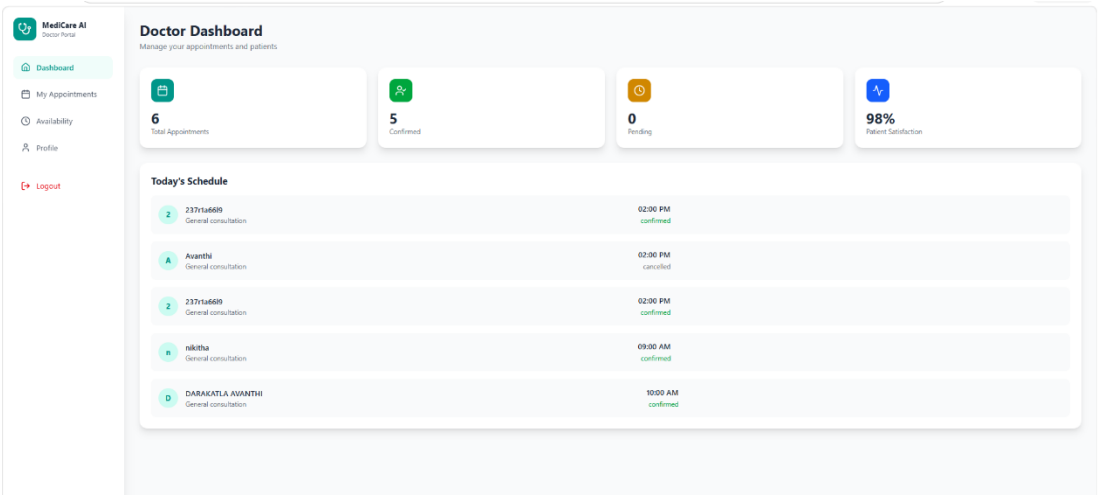
My Appointments

"This page displays all appointments, including upcoming, completed, and cancelled appointments, allowing patients to manage them easily."



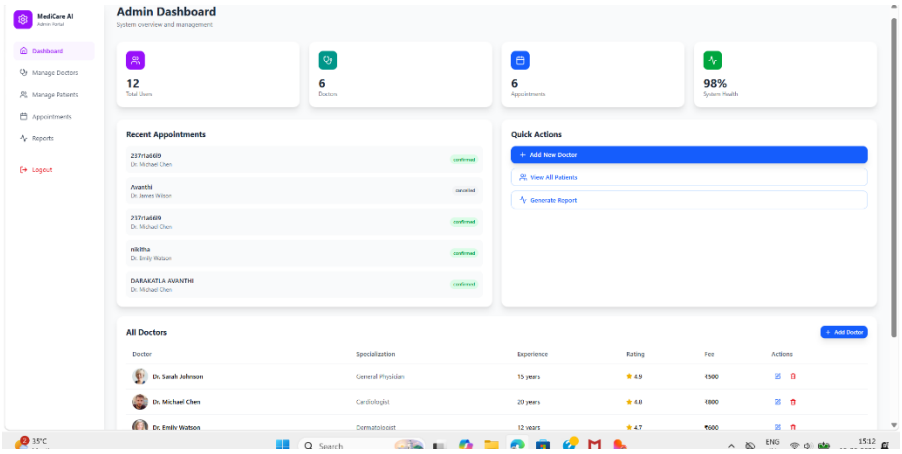
Doctor Login

"Doctors can securely access the system through this login page using their credentials."



Dashboard

"This dashboard helps doctors manage appointments, view schedules, monitor patient consultations, and track appointment statistics."



Admin Dashboard

"The admin dashboard provides complete control over the system, including managing doctors, patients, appointments, and monitoring overall platform activity."

CHAPTER – 6

CONCLUSION AND FUTURE SCOPE

Conclusion:

The AI Healthcare Appointment System provides an intelligent and efficient solution for healthcare appointment management. By combining web technologies and Machine Learning, the system helps patients identify the right doctor, schedule appointments online, and access healthcare services more conveniently. The project demonstrates the practical application of Artificial Intelligence in healthcare and contributes to improving patient experience and healthcare accessibility..

Future Scope:

The AI Healthcare Appointment System can be further enhanced by integrating advanced technologies and additional healthcare services. In the future, the system can support online video consultations, allowing patients to communicate with doctors remotely. Electronic Health Records (EHR) can be integrated to maintain complete medical histories and improve healthcare management. The AI module can be enhanced using advanced Machine Learning and Deep Learning algorithms to provide more accurate symptom analysis and disease prediction. A healthcare chatbot can be added to provide instant medical assistance and answer patient queries. The system can also include medicine recommendations, prescription management, laboratory test booking, and health monitoring features. Integration with wearable devices and IoT-based healthcare systems can enable real-time health tracking. Furthermore, multilingual support, mobile application development, cloud deployment, and hospital management system integration can expand the system's usability and accessibility, making it a comprehensive digital healthcare solution.

Future Enhancements

- Online Video Consultation with Doctors
- Electronic Health Records (EHR) Management
- Advanced AI-Based Disease Prediction
- AI Healthcare Chatbot Integration

- Medicine Recommendation System
- Online Prescription Management
- Laboratory Test Booking Facility
- Integration with Wearable Health Devices
- Mobile Application Development (Android & iOS)
- Cloud-Based Deployment and Scalability
- Multi-Language Support
- Hospital Management System Integration
- Real-Time Health Monitoring
- Emergency Healthcare Assistance Features
- Advanced Analytics and Reporting Dashboard

REFERENCES

Project Github Link :https://github.com/Avanthi38/AI-POWERED-HEALTHCARE-SYSTEM-AND-APPOINTMENT-SYSTEM